Graph Databases Explained

friend-of-friend, fraud rings and Cypher

Watch the 3-min video

> **In one line:** a graph database stores nodes and relationships and follows edges as pointer hops, so a query costs what it touches, not the size of the database. Choose it when your key queries traverse relationships many hops deep, or to a variable depth.

1 The problem: people you may know

People you may know: friends of your friends who aren't your friends yet. In SQL, that is a **self-join** on the friendships table, twice. Three hops means three joins, and each hop **multiplies the rows**. At social network scale, that query crawls. A graph database is built for exactly this.

2 The property graph model

A graph database stores **nodes** and **relationships**.

- **Nodes** are entities, like a person or an account, with **labels** (`:Person`, `:Account`).
- **Relationships** connect two nodes, with a **type** and a **direction**, like `FRIENDS_WITH` or `USES_DEVICE`.
- Both nodes and relationships can carry **properties** (key-value pairs).

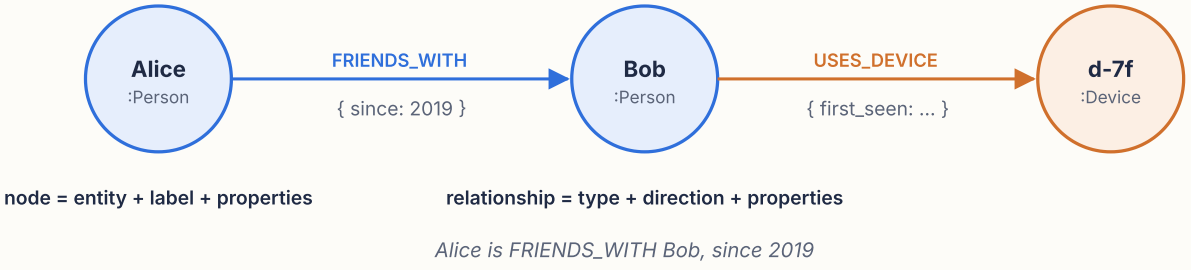


Figure 1. The property graph model: labelled nodes, typed and directed relationships, and properties on both.

3 Traversal: pointer hops, not joins

In Neo4j, each node **points directly to its relationships**, so following an edge is a **pointer hop**, not an index lookup on a giant table. The cost of a query depends on **how much of the graph you touch**, not on the total size of the database. This is called **index-free adjacency**. Indexes are still used, but only to **find the starting node**.

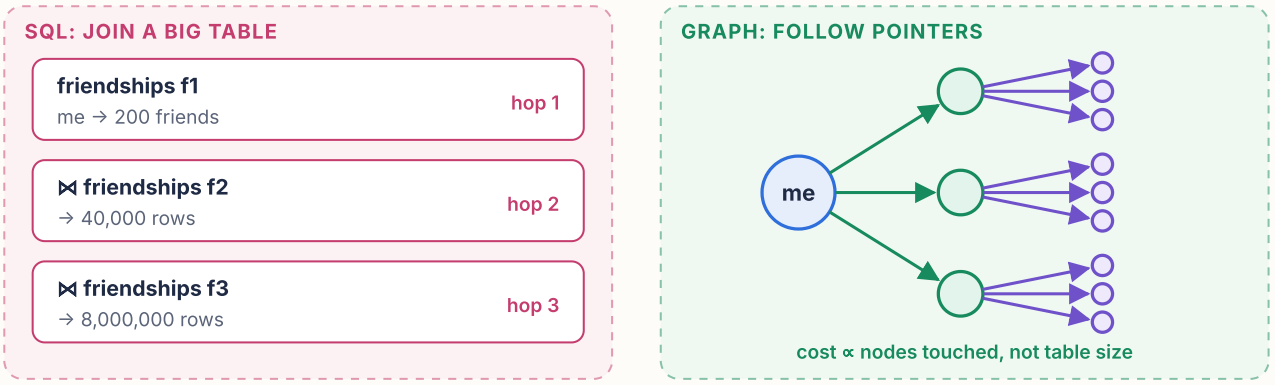


Figure 2. With 200 friends each, every SQL hop multiplies the rows by about 200, and each join probes an index on a table of billions of rows. The graph walks only the edges it needs.

WORKED EXAMPLE: WHY HOPS EXPLODE

- Each user has about **200 friends**. One hop: 200 rows. Two hops: 200 × 200 = **40,000**. Three hops: **8 million**.
- In SQL, each of those rows is an index lookup into a friendships table with, say, 100 billion rows. The work grows with the hops *and* the index depth.
- In a graph, the two-hop query visits about 40,000 edges, whether the database holds 1 million users or 1 billion.

4 Friend-of-friend in Cypher

Cypher is Neo4j's query language. Match me, my friends, and their friends. Exclude people I already know, and myself. Then count the mutual friends and return the top ten. The query reads like the picture you would draw on a whiteboard.

```
MATCH (me:Person {id: $userId})-[:FRIENDS_WITH]-(f:Person)-[:FRIENDS_WITH]-(fof:Person)
WHERE fof <> me
  AND NOT (me)-[:FRIENDS_WITH]-(fof)
RETURN fof.name, count(DISTINCT f) AS mutual
ORDER BY mutual DESC
LIMIT 10;
```

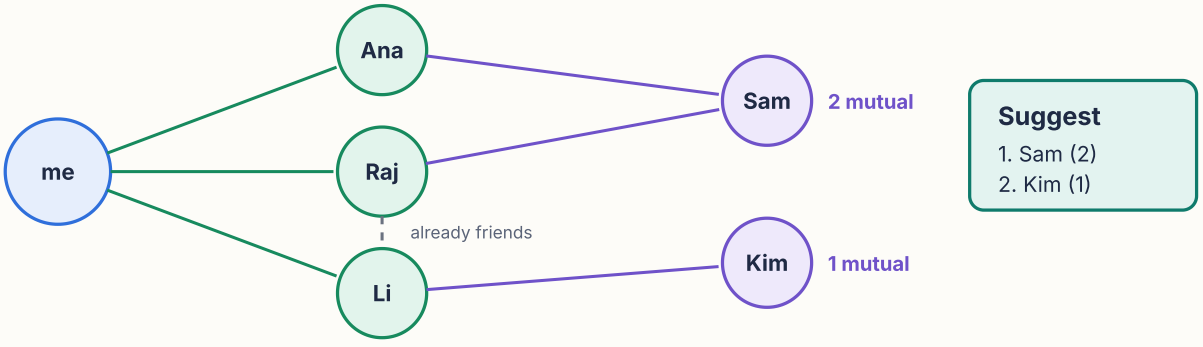


Figure 3. Sam is reached through two of my friends, Kim through one. Li is also a friend of Raj, but Li and Raj are already my friends, so they are excluded.

5 Fraud rings

Fraud rings are the other classic example. Accounts that **share a device, a phone number or a shipping address** form clusters. A **variable-length path query** finds every account within **four hops** of a known fraudster. In SQL, that is a recursive query, and it gets slow fast. Banks and payment companies use this to flag new accounts that connect to known fraud, **before money moves**.

```
MATCH (bad:Account {flagged: true})-[:USES_DEVICE|HAS_PHONE|SHIPS_TO*1..4]-(a:Account)
WHERE a <> bad
RETURN DISTINCT a.id;
```

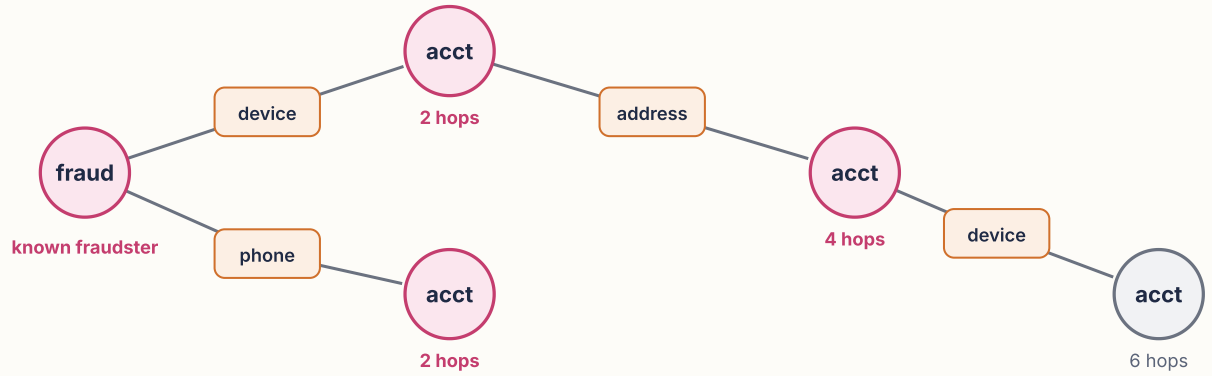


Figure 4. Account → shared device → account is 2 hops. Within 4 hops, three accounts are flagged for review; the one 6 hops away is not.

6 Systems, and when to choose a graph

- **Neo4j**, with Cypher.
- **Amazon Neptune**, a managed service supporting Gremlin, openCypher and SPARQL.
- **GQL** is now an ISO standard graph query language (published 2024).
- For simple hierarchies, like an org chart, **Postgres recursive queries** (`WITH RECURSIVE`) are often enough.

Choose a graph when relationships are the point: recommendations, fraud detection, network and dependency analysis, and knowledge graphs, especially with variable-depth queries. **Avoid it** for simple lookups, big aggregations across all records, or very high write volumes. Graphs are also **hard to shard**, because traversals cross partitions.

COMMON MISTAKES

- Using a graph database as the main store for simple records and reports; keep those relational.
- Unbounded variable-length paths (`*` with no limit) on a dense graph: the query explodes. Always cap the depth.
- No index on the start node's lookup property, so every query scans all nodes to begin.
- Ignoring super-nodes (a celebrity with 50 million followers) that every traversal passes through.

7 Key terms

Node: an entity with labels and properties

Relationship: a typed, directed edge with properties

Property graph: nodes + relationships + properties

Traversal: walking from node to node along edges

Index-free adjacency: each node points to its edges

Cypher: Neo4j's pattern query language

Variable-length path: *1..4: one to four hops

GQL: ISO standard graph query language

Super-node: a node with a huge number of edges

8 Interview questions

Q1. When would you choose a graph database? BASIC

When your most important queries traverse relationships many hops deep or with variable depth: friend-of-friend, fraud rings, dependency analysis, where SQL would need many expensive joins. Keep relational for simple records and aggregations.

Q2. What is index-free adjacency? BASIC

Each node stores direct pointers to its relationships, so following an edge is a pointer hop rather than an index lookup. Query cost depends on the part of the graph touched, not the total size.

Q3. Name the parts of the property graph model. BASIC

Nodes with labels (:Person), relationships with a type and direction (FRIENDS_WITH), and properties on both (since: 2019).

Q4. Design "people you may know" for 500 million users with ~200 friends each. SCENARIO

Store FRIENDS_WITH edges. For a user, traverse two hops (~40,000 paths), exclude existing friends and self, count mutual friends, return the top 10. Precompute nightly for most users and cache; run live for new users.

Q5. A new account shares a phone number with an account flagged for fraud last month. How do you catch it at signup? SCENARIO

On signup, write the account and its device, phone and address nodes, then run a bounded path query (*1..4) from the new account to any flagged account. A match within a few hops sends it to review before the first payment.

Q6. Your org chart has 20,000 employees and you need "everyone under this VP". Graph database? SCENARIO

Probably not. A Postgres WITH RECURSIVE query over an employees(id, manager_id) table handles 20,000 rows in milliseconds. A graph pays off when depth is variable and the data is large and densely connected.

Q7. Why are graph databases hard to shard? TRICKY

Traversals follow edges wherever they lead. Split the graph across machines and many hops become network calls, which kills the pointer-hop advantage. Real social graphs have no clean cut points.

Q8. A friend-of-friend query is fast for most users but times out for some. Why? TRICKY

Super-nodes. A user with 5,000 friends, some of whom are super-nodes with millions of edges, fans out to billions of paths. Cap fan-out per hop, sample, skip super-nodes, or precompute their results.

Next: SD-E-018 · Time-Series Databases backendroadmap.com